NEXT WORD PREDICTION USING LSTM NETWORKS

Objective: Implement an LSTM-based language model that learns from text sequences and predicts the next word from a given input phrase.

1. Understanding Next-Word Prediction

Next-word prediction is a language-modelling task. The model receives a sequence of words and learns to predict the word that should come next. For example, from the sentence **“Artificial Intelligence is transforming the world”**, it creates training examples where each input is a partial sequence and the target is the next word.

Input sequence	Expected next word
Artificial	Intelligence
Artificial Intelligence	is
Artificial Intelligence is	transforming
Artificial Intelligence is transforming	the
Artificial Intelligence is transforming the	world

2. Execution Workflow

Stage	Description
Input text corpus	Load review text from sentiment_dataset.csv. Sentiment labels are not used.
Tokenization	Split text into words and build a vocabulary that maps words to integer IDs.
Sequence generation	Create progressively longer word sequences and use the final word as the target.
Padding	Make input sequences equal in length so they can be processed in batches.
Embedding layer	Convert integer word IDs into dense numerical vectors.
LSTM layer	Learn sequence patterns and relationships between earlier and later words.
Dense + Softmax	Estimate a probability for every word in the vocabulary.
Text generation	Append the predicted word and repeat the process to generate more words.

3. Software Requirements

Use Python with TensorFlow/Keras, pandas, NumPy and Matplotlib. CPU execution is sufficient for this small demonstration.

```
python -m pip install --upgrade pip
pip install tensorflow pandas numpy matplotlib
```

4. Understanding the Main Concepts

A. Tokenization and vocabulary

Tokenization splits a sentence into individual words. A vocabulary is then created and each word is assigned an integer ID. For example, “the product is excellent” becomes a sequence of word IDs such as **[4, 12, 7, 25]**. The IDs depend on the vocabulary built from the training corpus.

B. Sequence generation

For each sentence, the program creates multiple training examples. Each example contains the previous words as input and the next word as the target.

Input (words)	Target
Artificial	Intelligence
Artificial Intelligence	is
Artificial Intelligence is	transforming
Artificial Intelligence is transforming	the
Artificial Intelligence is transforming the	world

C. Padding

Training sequences have different lengths. Padding adds zero values at the beginning of shorter sequences so that all inputs have the same length. This is called pre-padding.

Original sequence	After pre-padding to length 5
[8, 14]	[0, 0, 0, 8, 14]
[8, 14, 21]	[0, 0, 8, 14, 21]
[8, 14, 21, 6]	[0, 8, 14, 21, 6]

D. Embedding layer

The Embedding layer converts each integer word ID into a trainable vector. In this experiment, each word is represented by 64 numerical values. The model learns these vector representations during training.

E. LSTM and output layer

The LSTM learns patterns across the input sequence. The final Dense layer has one output unit for each word in the vocabulary. Softmax converts the output scores into probabilities, and the word with the highest probability is selected as the next word.

5. Model Architecture

Layer	Role
Input	Receives a fixed-length sequence of integer word IDs.
Embedding (64)	Maps each word ID to a 64-dimensional vector.
LSTM (100 units)	Learns sequential patterns from the embedded words.
Dense (vocabulary size)	Produces one score for every word in the vocabulary.
Softmax	Converts scores into a probability distribution over the vocabulary.

Important distinction: This is a language model, not a sentiment classifier. It learns to predict the next word and does not predict Positive or Negative labels.

6. Complete Python Implementation (Part 1)

Create a file named **lstm_next_word_prediction.py** in the same folder as **sentiment_dataset.csv**. Add the following code.

```
# NEXT WORD PREDICTION USING LSTM NETWORKS

import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.callbacks import EarlyStopping

# STEP 1: Load text dataset
data = pd.read_csv("sentiment_dataset.csv")

if "Review" not in data.columns:
    raise ValueError("CSV must contain a Review column.")

texts = data["Review"].dropna().astype(str).tolist()

if not texts:
    raise ValueError("No review text found in the dataset.")

np.random.seed(42)
tf.random.set_seed(42)

print("Total sentences:", len(texts))

# STEP 2: Tokenization and vocabulary generation
MAX_WORDS = 5000
MAX_TEXT_LENGTH = 50

vectorizer = tf.keras.layers.TextVectorization(
    max_tokens=MAX_WORDS,
    standardize="lower_and_strip_punctuation",
    split="whitespace",
    output_mode="int",
    output_sequence_length=MAX_TEXT_LENGTH
)

# Learn the vocabulary from the text corpus
vectorizer.adapt(texts)

vocabulary = vectorizer.get_vocabulary()
total_words = len(vocabulary)

print("\\nVocabulary size:", total_words)

# STEP 3: Generate next-word training sequences
encoded_sentences = vectorizer(
    tf.constant(texts)
).numpy()

input_sequences = []

for sentence in encoded_sentences:
    # Remove padding tokens
    token_list = [
        int(token) for token in sentence if token != 0
    ]

    # Create progressively longer sequences
    for i in range(1, len(token_list)):
        ngram_sequence = token_list[:i + 1]
        input_sequences.append(ngram_sequence)

if not input_sequences:
    raise ValueError("Not enough words to create training sequences.")

print("Total training sequences:", len(input_sequences))
```

6. Complete Python Implementation (Part 2)

Continue the same Python file with the following code:

```
# STEP 4: Padding
max_sequence_len = max(
    len(sequence) for sequence in input_sequences
)

padded_sequences = tf.keras.utils.pad_sequences(
    input_sequences,
    maxlen=max_sequence_len,
    padding="pre"
)

print("Maximum sequence length:", max_sequence_len)

# STEP 5: Separate input and target
X = padded_sequences[:, :-1]
y = padded_sequences[:, -1]

print("Input shape:", X.shape)
print("Target shape:", y.shape)

# STEP 6: Build LSTM architecture
model = tf.keras.Sequential([
    tf.keras.Input(
        shape=(max_sequence_len - 1,),
        dtype="int32"
    ),

    # Embedding layer
    Embedding(
        input_dim=total_words,
        output_dim=64,
        mask_zero=True
    ),

    # LSTM layer
    LSTM(units=100),

    # Predict a probability for each vocabulary word
    Dense(total_words, activation="softmax")
])

# STEP 7: Compile model
model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam",
    metrics=["accuracy"]
)

print("\\nMODEL ARCHITECTURE")
model.summary()

# STEP 8: Train the model
early_stop = EarlyStopping(
    monitor="loss",
    patience=10,
    restore_best_weights=True
)

print("\\nTraining LSTM model...")
history = model.fit(
    X,
    y,
    epochs=100,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

print("\\nTraining completed!")
```

6. Complete Python Implementation (Part 3)

Continue the same Python file:

```
# STEP 9: Next-word prediction function
index_to_word = {
    index: word for index, word in enumerate(vocabulary)
}

def generate_text(seed_text, next_words=5):

    for _ in range(next_words):

        # Convert the seed phrase into word IDs
        token_list = vectorizer(
            tf.constant([seed_text])
        ).numpy()[0]

        token_list = [
            int(token) for token in token_list
            if token != 0
        ]

        # Pad input sequence to the model input length
        token_list = tf.keras.utils.pad_sequences(
            [token_list],
            maxlen=max_sequence_len - 1,
            padding="pre",
            truncating="pre"
        )

        # Predict probabilities for the next word
        predicted_probs = model.predict(
            token_list, verbose=0
        )[0]

        # Avoid padding and unknown tokens
        predicted_probs[0] = 0.0
        if total_words > 1:
            predicted_probs[1] = 0.0

        predicted_index = int(
            np.argmax(predicted_probs)
        )

        # Convert predicted ID into a word
        predicted_word = index_to_word.get(
            predicted_index, ""
        )

        if not predicted_word:
            break

        # Append predicted word
        seed_text += " " + predicted_word

    return seed_text

# STEP 10: Generate text
print("\\nNEXT WORD PREDICTION")
seed_text = "the product"
print("Input text:", seed_text)

generated_text = generate_text(
    seed_text, next_words=5
)

print("\\nGenerated text:")
print(generated_text)
```

6. Complete Python Implementation (Part 4)

Add the final training-curve and model-saving section:

```
# STEP 11: Plot training loss
plt.figure(figsize=(7, 4))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.title("LSTM Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# STEP 12: Save trained model
model.save("next_word_lstm.keras")

print("\\nModel saved successfully!")
```

7. How to Execute the Program

1. Create a folder named **LSTM_Next_Word_Prediction**.

2. Place **sentiment_dataset.csv** and **lstm_next_word_prediction.py** inside the folder.

3. Open Command Prompt, PowerShell or the VS Code terminal.

4. Navigate to the project folder using the cd command.

5. Install TensorFlow, pandas, NumPy and Matplotlib.

6. Run the Python program and observe vocabulary size, sequence count, training progress and generated text.

Terminal commands

```
cd path\to\LSTM_Next_Word_Prediction
python -m pip install --upgrade pip
pip install tensorflow pandas numpy matplotlib
python lstm_next_word_prediction.py
```

8. Important Parameters

Parameter	Explanation
MAX_WORDS = 5000	Maximum vocabulary size retained by TextVectorization.
MAX_TEXT_LENGTH = 50	Maximum number of tokens retained for each source sentence.
output_dim = 64	Each word is represented by a 64-dimensional embedding vector.
LSTM units = 100	Number of LSTM units used to process the sequence.
Dense(total_words)	Creates one output score for each vocabulary entry.
softmax	Converts output scores into probabilities over the vocabulary.
epochs = 100	Maximum training passes; early stopping may stop training earlier.
batch_size = 32	Number of training examples processed per batch.

9. Understanding the Output

Output	Meaning
Vocabulary size	Number of tokens learned by the vocabulary layer, including reserved tokens.
Training sequences	Number of input-target examples generated from source sentences.
Training loss	Measures how far the predicted next-word distribution is from the target word.
Training accuracy	Fraction of examples where the highest-probability word matches the target.
Generated text	The seed phrase followed by words predicted one at a time.
Saved model	next_word_lstm.keras contains the trained model.

Important: The actual generated words and scores depend on model training. The sample corpus is small, so output may be repetitive, ungrammatical or unrelated to the seed phrase.

10. Understanding the Model Architecture

Layer	Role
Input	Receives a fixed-length sequence of integer word IDs.
Embedding	Maps each word ID to a 64-dimensional numerical vector.
LSTM	Learns sequence patterns and dependencies between words.
Dense	Produces a score for every word in the vocabulary.
Softmax	Converts scores into a probability distribution; the highest probability is selected.

11. Suggested Laboratory Experiment

Title: Next Word Prediction using LSTM Networks.

Aim: To implement an LSTM-based next-word prediction model using tokenization, sequence generation, padding and embedding layers, and generate text based on a given input sequence.

Learning Outcomes

1. Understand tokenization and vocabulary generation.
2. Generate sequential training data for next-word prediction.
3. Apply padding to create fixed-length input sequences.
4. Implement an Embedding and LSTM network.
5. Generate text by predicting one word at a time.

12. Teaching Sequence

Order	Classroom demonstration
1	Tokenize a sentence and explain the vocabulary and integer IDs.
2	Generate input sequences and their next-word targets.
3	Show how padding makes the sequences equal in length.
4	Explain how the Embedding and LSTM layers process the input.
5	Train the model and demonstrate next-word generation from a seed phrase.

13. Dataset Recommendation and Limitations

The earlier sentiment_dataset.csv contains only 100 short synthetic reviews. It is sufficient to demonstrate tokenization, sequence generation, padding and LSTM execution, but it is not a suitable corpus for coherent language generation. A stronger experiment should use a much larger continuous text corpus such as articles, paragraphs or books. Sentence-level reviews provide limited long-range context.

14. Documentation

Keras LSTM: keras.io - LSTM
Keras Embedding: keras.io - Embedding
Keras sequence padding: keras.io - sequence utilities
TensorFlow installation: tensorflow.org/install/pip